What Makes a VLA Reactive? A Study of Execution Strategies for Dynamic Manipulation

Zhuoyue Li[1], Zhendong Han[1], Pankhuri Vanjani[1], Wenting Li[1], Rudolf Lioutikov[1,2]

[1]Intuitive Robots Lab (IRL), Karlsruhe Institute of Technology (KIT)
[2]Robotics Institute Germany (RIG)

Abstract: Vision-language-action (VLA) models predict chunks of future actions, and when objects move, those chunks go stale before they are executed. We study what makes a pretrained VLA reactive, using $\pi_{0.5}$ on two dynamic simulation benchmarks and three real-robot tasks. We vary the prediction and execution horizons, compare synchronous and asynchronous inference, test methods that deliver actions sooner or keep successive chunks continuous, and adapt the policy with motion cues and latent-space reinforcement learning. The best prediction horizon is about one second, and the best execution horizon depends on the checkpoint. Short horizons help more with asynchronous execution, and methods that keep the trajectory continuous outperform those that only reduce latency. Adapting the policy repairs targeted failures, but on the conveyor task changing the execution horizon gained as much. Reactivity depends on the execution pipeline as much as on the policy, so both should be evaluated together.

1 Introduction

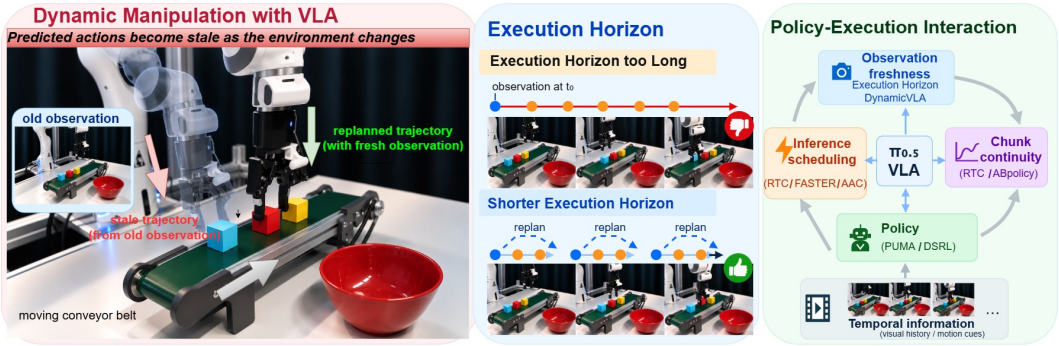


Figure 1: Dynamic VLA manipulation depends on the interaction between prediction and execution horizons, inference scheduling, chunk continuity, and policy behavior, which together determine whether frequent replanning improves task performance.

Vision-Language-Action (VLA) models predict action chunks to support coherent motion with fewer inference calls. When objects move during inference and execution, these predictions can become stale. Shorter execution horizons allow more frequent replanning, but increase inference demand and can introduce discontinuities between chunks. Dynamic manipulation therefore requires balancing observation freshness, computational cost, and execution stability. Recent approaches address these challenges through faster action generation, asynchronous execution, trajectory continuity, and temporal information [1, 2, 3, 4]. We used $\pi_{0.5}$ [5] on DOM [3], DOMINO [4], and three real-robot dynamic tasks. In this work, we investigate: **(1)** How do prediction and execution horizons affect performance? **(2)** How do scheduling and trajectory continuity affect the benefits of frequent replanning? **(3)** Can temporal information and reward-guided adaptation, including DSRL [6], improve dynamic manipulation? Our results show that frequent replanning alone does not ensure better performance: its benefits depend on the execution pipeline and the learned policy.

10th Conference on Robot Learning (CoRL 2026), Austin TX, USA.

2 Study Design

Problem Setup Given visual observations o_t, language instruction ℓ, and proprioceptive state p_t, the policy predicts an action chunk $A_t = (a_{t|t}, \ldots, a_{t+H-1|t}) \sim \pi_\theta(\cdot \mid o_t, \ell, p_t)$, where H is the *prediction horizon*. The *execution horizon* $s \leq H$ specifies how many actions are executed from a predicted chunk. Thus, prediction length and execution commitment can be varied separately [1, 2]. In *synchronous execution*, the controller executes the selected actions, then waits for the next prediction. *Asynchronous execution* overlaps inference with ongoing motion, reducing this waiting time. Inference latency τ corresponds to a delay of d action steps. Asynchronous execution must account for motion during this delay when selecting or conditioning the next chunk. Switching between independently generated chunks can also introduce discontinuities in the commanded trajectory [1]. We therefore study execution horizons, inference scheduling, and continuity, and whether temporal information or policy adaptation can compensate for these effects.

Experimental Setup We evaluate $\pi_{0.5}$-based configurations on DOM [3] and DOMINO [4] simulation tasks and three real-robot tasks using a Franka Panda: *moving-cup placement*, *rolling-can grasping*, and *conveyor-belt picking*. The conveyor task uses three cubes with a red target cube under stationary, slow ($2.5\,\mathrm{cm/s}$), and fast ($5\,\mathrm{cm/s}$) belt conditions. Left, Middle, and Right denote the target's position along the stationary belt or its relative position within the moving three-cube sequence. Training, hardware, control rates, task configurations, and evaluation protocols are detailed in the appendix.

2.1 Intervention Selection and Implementation

We select approaches that address complementary limitations of dynamic manipulation: temporal commitment, delayed or discontinuous execution, and insufficient motion information or behavioral adaptation. We implement these interventions within a common $\pi_{0.5}$ framework. Table 1 summarizes their intervention points. These configurations adapt selected mechanisms from the original methods rather than reproduce every original system.

Table 1: Intervention levels and goal of evaluated methods.

Method	Model	Inference	Execution	Action Rep.	Target
RTC		✓			Chunk transition
FASTER		✓	✓		Action generation
DynamicVLA		✓	✓		Execution alignment
ABPolicy	✓		✓	✓	Chunk continuity
PUMA	✓				Motion information
AAC		✓	✓		Execution horizon
DSRL			✓		Latent steering

Temporal Horizons and Replanning The prediction horizon H sets how far ahead the policy plans, and the execution horizon s sets how much of that plan is carried out before the policy looks again. Together they decide how old the observation behind each executed action can be. We vary H during fine-tuning and s at deployment with a fixed policy. A fixed s treats every moment the same, so we also test whether the policy can choose s itself with Adaptive Action Chunking (AAC) [7]. AAC samples several candidate chunks, measures the entropy of the predicted actions, and ends a chunk early when the entropy is high, subject to a minimum action-magnitude constraint in Cartesian space.

Scheduling and Continuity Interventions A shorter s only helps if new actions arrive in time and join the old ones smoothly. We therefore compare synchronous and asynchronous execution across execution horizons, and evaluate methods that address each requirement as separate $\pi_{0.5}$ configurations: FASTER and DynamicVLA make actions available sooner, RTC and ABPolicy keep the trajectory continuous across chunks. Our DynamicVLA adaptation uses continuous inference and latency-aware action skipping, adding a minimum execution window while retaining the $\pi_{0.5}$ architecture and inputs [3]. With FASTER, we get near-term actions earlier through horizon-aware sampling, action streaming, and early stopping [2]. To test whether continuity-aware chunk transitions improve dynamic performance, we evaluate RTC and ABPolicy. RTC uses inference-time inpainting to align new predictions with overlapping actions from the previous chunk without additional policy training [1]. ABPolicy combines bidirectional prediction with continuity-constrained spline refitting. We adapt it to Cartesian actions and train the policy to predict spline control points [8].

Motion Information and Policy Adaptation The interventions above change how the policy is executed, the last two change the policy itself. PUMA gives it information about how the object moves, and DSRL adjusts what it does with that information. PUMA incorporates historical optical flow and future object-centric feature supervision during training [4]. DSRL trains a latent-space actor and critic while keeping the underlying VLA frozen [6]. We use DSRL-SAC with a short visual history aggregated by a temporal convolution for the actor and critic, while the VLA retains its current-observation input.

3 Experimental findings and Discussion

We organize our evaluation around three research questions: **RQ1:** How do prediction and execution horizons affect dynamic manipulation performance? **RQ2:** How do inference scheduling and trajectory continuity affect the benefits of frequent replanning? **RQ3:** Can temporal information and reward-guided adaptation improve dynamic manipulation?

Table 2: **RQ1: How do prediction and execution horizons affect dynamic manipulation?**

Finding	Result and explanation
Training: a prediction horizon of about one second works best.	The best prediction horizon is 25 steps on DOM and 30 steps on DOMINO. At their control rates of 25 and 30 Hz, both are one second of motion (Fig. 2a). With longer chunks, the robot keeps executing a plan made for an object that has since moved on.
Deployment: the best execution horizon depends on the checkpoint, not on replanning more often.	The 1k checkpoint works best at $s{=}5$ and the 5k checkpoint at $s{=}15$ (Fig. 2b). The 5k policy barely moves during the first steps of each chunk. If we replan sooner, the robot sees almost the same scene, predicts the same slow start again, and never gets going. Replanning helps only when the new prediction is different from the old one.
Adaptive horizons do not help.	AAC never beats the best fixed horizon and fails on the static and fast belts (Fig. 2c). It samples several candidate chunks to measure the entropy of the predicted actions, and this extra inference delays the next chunk. *Entropy is the wrong trigger here.* It is high when the scene is unfamiliar or when several actions are equally good, and in both cases a new image does not change the decision. It is low when the policy is confident, even if the object is about to move away. AAC therefore replans at the wrong moments and pays extra inference time to do so.

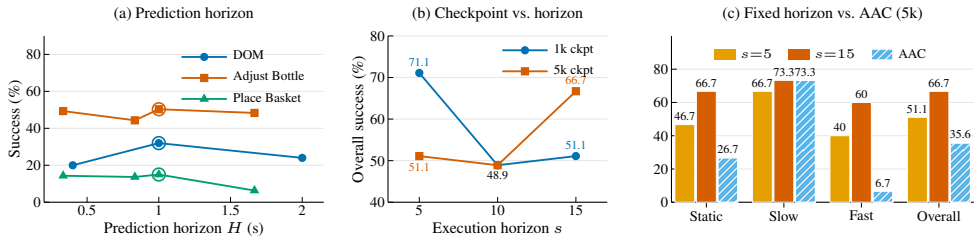


Figure 2: **RQ1.** (a) Success against the prediction horizon in seconds on DOM and DOMINO; rings mark the best horizon. (b) Conveyor-belt success of the 1k and 5k checkpoints across execution horizons. (c) Success of the 5k checkpoint with fixed horizons and AAC per belt condition ($H{=}20$, asynchronous, 45 trials each).

Table 3: **RQ2: When does more frequent replanning actually help?**

Finding	Result and explanation
Scheduling: short horizons help more with asynchronous execution.	At $s{=}10$, the synchronous $\pi_{0.5}$ and PUMA policies succeed in only 20% of trials, while the asynchronous methods reach 60–75% (Fig. 3c). On DOM the gap grows as the horizon shrinks (Fig. 3a). In synchronous execution, the robot finishes a chunk and then waits for the policy to compute the next one; during that wait it does not move. A shorter horizon means more chunks, so more waits, and the object moves on during each of them. In asynchronous execution the next chunk is computed while the current one is still running, so a short horizon adds no waiting.
Continuity: methods that preserve continuity lead, and low latency alone does not.	ABPolicy and RTC are the best methods on the conveyor task, although FASTER has the lowest latency (Fig. 3c, Table 5). A fresh action is only useful if the robot can follow it. When a new chunk starts far from where the old one ended, the robot has to jump, and that costs more than the faster inference saves.
Action rate: on the fast belt, every method improves at 30 Hz.	Raising the action rate from 20 to 30 Hz lifts ABPolicy from 13 to 93% and FASTER from 0 to 33% on the fast belt (Fig. 3b). The same chunk is finished sooner, so the object has less time to move away before the next observation.

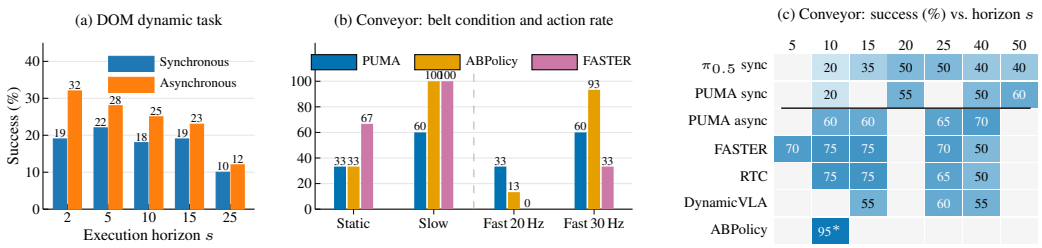


Figure 3: **RQ2.** (a) Success on the dynamic DOM task with the same policy (H=25) under synchronous and asynchronous execution. (b) Conveyor-belt success of the best configuration of each method on the static, slow, and fast belt, and on the fast belt at a higher action rate; each bar is 15 trials. (c) Conveyor-belt success of every method across execution horizons, with synchronous methods above the rule. Blank cells were not evaluated; ABPolicy (*) was evaluated only at $s = 9$, as its effective execution horizon depends on inference delay

Table 4: **RQ3: Can temporal information or reward-guided adaptation make the policy itself more reactive?**

Finding	**Result and explanation**
Motion information: PUMA gives mixed results.	PUMA does better than $\pi_{0.5}$ on the conveyor but worse on the rolling can (Fig. 4a). Motion cues help on the conveyor, and asynchronous execution makes them count: run synchronously, PUMA still fails at short horizons like the baseline, but run asynchronously it reaches 60–70% at every horizon (Fig. 3c). Knowing where the object is going is only useful when the policy also acts on fresh chunks.
Latent steering: DSRL repairs targeted failures without touching the VLA.	On the targeted condition, success rises from 32 to 47% on DOM and from 0 to 60% on the hardest conveyor case (Fig. 4b). The gain depends on the checkpoint and disappears at a large noise magnitude. DSRL only changes the noise that the frozen policy turns into actions, so it can reach a behaviour only if the policy already contains it.
Scope of steering: the gains stay local and match what a change of execution horizon gives for free.	Averaged over all nine conditions, DSRL changes success by 4 to 20 points and some conditions get worse (Fig. 4c). It lifts the 1k checkpoint to 64–69%, but the same checkpoint reaches 71% at s=5 with no training at all (Fig. 2b). Changing how the policy is executed was worth as much as training it.

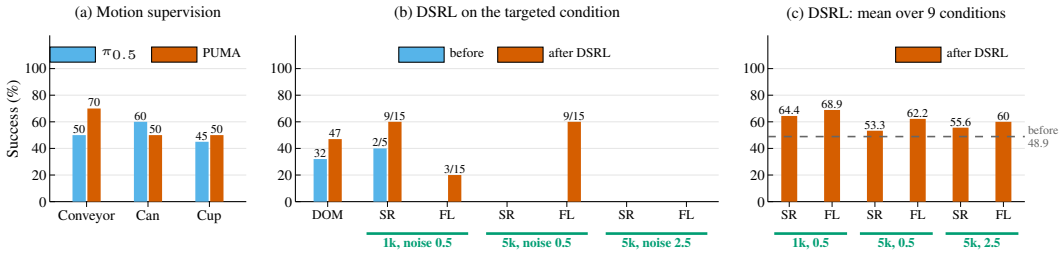


Figure 4: **RQ3.** (a) Best success of $\pi_{0.5}$ and PUMA on each real-robot task. (b) Success on the targeted condition before and after DSRL, on DOM in percent and on the conveyor belt as success counts over 5 trials before and 15 after; brackets give the checkpoint and the noise magnitude. (c) Mean success over all nine conveyor conditions after DSRL; the dashed line is the 48.9% of the original checkpoints.

4 Conclusion and Limitations

We asked what makes a pretrained VLA reactive. In our experiments, much of the answer lies in how the policy is executed. The prediction horizon is best chosen in seconds, and the best execution horizon depends on the checkpoint. A shorter horizon only helped when the new prediction actually changed the action. Short horizons worked well when inference ran alongside execution and when successive chunks joined smoothly. Lower latency alone did not help. Motion cues and latent steering helped in targeted cases. On the conveyor task, however, a different execution horizon gained as much without any training. We suggest evaluating the execution strategy together with the policy, and reporting observation age, action availability, and chunk continuity alongside success.

Our findings characterise $\pi_{0.5}$-based adaptations of the evaluated methods on a small set of tasks. Real-robot comparisons use 15–20 trials per setting, and the can and cup tasks rely on manually induced object motion, so small differences should not be over-interpreted. Adaptive horizon strategies should weigh the expected benefit of replanning, inference overhead, and trajectory continuity, not action uncertainty alone.

References

[1] K. Black, M. Galliker, and S. Levine. Real-time execution of action chunking flow policies. In *Advances in Neural Information Processing Systems*, volume 38, 2025.

[2] Y. Lu, Z. Liu, X. Fan, Z. Yang, J. Hou, J. Li, K. Ding, and H. Zhao. Faster: Rethinking real-time flow vlas, 2026. URL https://arxiv.org/abs/2603.19199.

[3] H. Xie, B. Wen, J. Zheng, Z. Chen, F. Hong, H. Diao, and Z. Liu. Dynamicvla: A vision-language-action model for dynamic object manipulation, 2026. URL https://arxiv.org/abs/2601.22153.

[4] H. Fang, S. Li, S. Wang, X. Xi, D. Liang, and X. Bai. Towards generalizable robotic manipulation in dynamic environments, 2026. URL https://arxiv.org/abs/2603.15620.

[5] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, M. Y. Galliker, D. Ghosh, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, D. LeBlanc, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, A. Z. Ren, L. X. Shi, L. Smith, J. T. Springenberg, K. Stachowicz, J. Tanner, Q. Vuong, H. Walke, A. Walling, H. Wang, L. Yu, and U. Zhilinsky. $\pi_{0.5}$: a vision-language-action model with open-world generalization, 2025. URL https://arxiv.org/abs/2504.16054.

[6] A. Wagenmaker, M. Nakamoto, Y. Zhang, S. Park, W. Yagoub, A. Nagabandi, A. Gupta, and S. Levine. Steering your diffusion policy with latent space reinforcement learning. *Conference on Robot Learning*, 2025.

[7] Y. Liang, X. Wang, K. Wang, S. Wang, X. Peng, H. Chen, D. K. H. Chua, and P. Vadakkepat. Adaptive action chunking at inference-time for vision-language-action models, 2026. URL https://arxiv.org/abs/2604.04161.

[8] F. Yang, P. Jing, K. Qu, N. Zhao, and Y. Su. Abpolicy: Asynchronous b-spline flow policy for real-time and smooth robotic manipulation, 2026. URL https://arxiv.org/abs/2602.23901.

[9] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Robotics: Science and Systems*, 2023.

[10] Y. Zhao, M. Bogdanovic, A. Sohal, L. Tao, K. Darvish, A. Aspuru-Guzik, F. Shkurti, and A. Garg. Dynamic execution horizon prediction for chunk-based robot policies, 2026. URL https://arxiv.org/abs/2606.11408.

[11] Y. Chen, K. Yu, P. Liu, S. Wang, Y. Yang, and L. Zhu. Ravel: Asynchronous rolling inference for flow-based vision-language-action models, 2026. URL https://arxiv.org/abs/2609.34170.

[12] K. Hosseinkhani, Q. Peng, G. Shramko, M. Aghabozorgi, J. Qian, T. Engst, A. Moazeni, D. Jayaraman, and K. Li. Imle-vla: Fast single-step action generation for vision-language-action policies. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2026. URL https://api.semanticscholar.org/CorpusID:291902373.

[13] C. Yang, Y. Hu, Y. Ma, Y. Yang, J. Tan, and H. Fan. Realtime-vla v2: Learning to run vlas fast, smooth, and accurate, 2026. URL https://arxiv.org/abs/2603.26360.

[14] K. Black, A. Z. Ren, M. Equi, and S. Levine. Training-time action conditioning for efficient real-time chunking, 2025. URL https://arxiv.org/abs/2512.05964.

[15] T.-B. Ho, Q.-T. Nguyen, T.-L. Ha, G.-B. Nguyen, V.-T. Nguyen, L. Dinh, M. N. Vu, D. M. Nguyen, A. T. Le, and N. A. Vien. Start right, arrive right: Asynchronous execution via initial noise selection. In *Conference on Robot Learning (CoRL)*, 2026.

[16] Y. Hu, J.-N. Zaech, N. Nikolov, Y. Yao, S. Dey, G. Albanese, R. Detry, L. V. Gool, and D. Paudel. Ar-vla: True autoregressive action expert for vision-language-action models, 2026. URL https://arxiv.org/abs/2603.10126.

[17] Y. Shin, J. Chae, S. Kim, J. Park, and Y. Sung. Adaptive action chunking via multi-chunk q value estimation, 2026. URL https://arxiv.org/abs/2605.10044.

[18] P. Dong, K.-H. Hung, D. Sadigh, and C. Finn. Reinforcement learning for real-time vision-language-action policies, 2026. URL https://arxiv.org/abs/2609.18207.

Appendix

A Related Work

A.1 Action Chunking and Execution Horizons

Action chunking has become a common design choice in modern robot policies, including many Vision-Language-Action (VLA) models. Rather than predicting a single action at each timestep, a chunk-based policy predicts a sequence of future actions jointly, an idea popularized in robotic imitation learning by Action Chunking with Transformers (ACT) [9]. This formulation provides a temporally structured action representation and reduces the need to make each control decision independently. Modern VLA systems retain this basic interface while combining it with large-scale visual, language, and robot representations. For example, $\pi_{0.5}$ uses a flow-based action expert to generate continuous robot action sequences from multimodal observations [5].

The prediction horizon determines how many future actions are generated from a single observation, whereas the execution horizon specifies how many actions are executed before replanning. In practice, the execution horizon can be substantially shorter than the prediction horizon, allowing the controller to update its predictions using new observations [1, 10, 2]. Shorter execution horizons increase replanning frequency and can improve closed-loop responsiveness, but also increase inference demand and the frequency of transitions between independently generated action chunks. The appropriate horizon can additionally depend on the action representation.

A.2 Real-Time Action Generation and Inference Scheduling

In synchronous execution, the robot completes one action chunk before requesting the next, introducing pauses between chunks. Asynchronous execution overlaps action generation with ongoing motion, reducing idle time but introducing new timing challenges. As shown in Figure 5, actions may become outdated during inference, while switching between independently generated chunks can introduce trajectory discontinuities.

DynamicVLA addresses inference delay and outdated actions through Continuous Inference (CI) and Latency-Aware Action Streaming (LAAS) [3]. CI overlaps inference and execution, while LAAS discards outdated actions and prioritizes newer predictions when chunks overlap.

FASTER shifts the focus from full-chunk completion time to *time to first action* (TTFA) [2]. Its Horizon-Aware Schedule (HAS) generates near-term actions with fewer denoising steps, allowing them to be streamed before the entire chunk is finalized. RAVEL instead uses rolling action generation and asynchronous VLM encoding to maintain partially denoised future actions and incorporate recent visual information through a lightweight observation pathway [11].

A complementary approach is IMLE-VLA, which replaces iterative action sampling with a single-step conditional generator to reduce inference latency [12]. Beyond inference, Realtime-VLA V2 addresses sensing delays, trajectory feasibility, and task-dependent execution speed to improve physical robot execution [13].

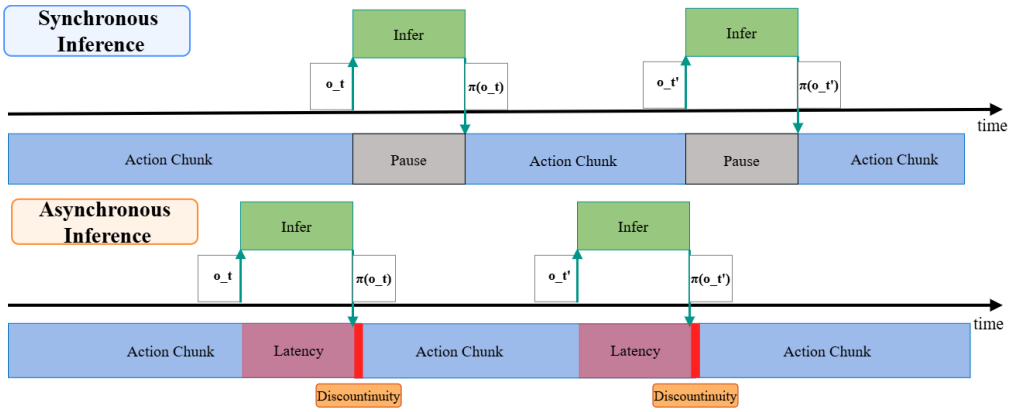


Figure 5: Synchronous execution introduces waiting periods, while asynchronous execution overlaps inference with motion but may cause inter-chunk discontinuities.

These methods improve action availability and temporal alignment through different mechanisms. However, faster action generation alone does not guarantee better closed-loop responsiveness, as execution timing and trajectory continuity remain important.

A.3 Trajectory Continuity and Action Representation

Asynchronous execution can introduce discontinuities when switching between independently generated action chunks. Real-Time Chunking (RTC) addresses this problem through inference-time action inpainting [1]. It uses soft masking to preserve near-term continuity while allowing future actions to adapt to new observations. RTC requires no retraining but introduces additional computation through gradient-based guidance.

Training-Time Action Conditioning extends RTC by learning prefix conditioning during training instead of applying inference-time guidance, reducing inference overhead at the cost of additional training [14]. PAINT addresses the same prefix-consistency problem through initial noise selection rather than modifying the denoising process [15]. AR-VLA takes a different approach by using an autoregressive action expert with persistent action history to maintain temporal consistency across predictions [16]. These methods address temporal consistency at different levels: inference-time guidance, training-time conditioning, noise initialization, and persistent action history.

From the action-representation perspective, ABPolicy combines asynchronous inference with a B-spline control-point representation [8]. Cubic B-splines provide C^2 continuity within each chunk. To reduce inter-chunk discontinuities, ABPolicy combines Bidirectional Action Prediction (BiAP), which incorporates past actions into prediction, with Continuity-Constrained Refitting (CCR), which aligns the new trajectory with recently executed actions.

A.4 Dynamic Manipulation and Temporal Reasoning

Dynamic manipulation introduces additional challenges because objects may move independently of the robot. A predicted action sequence can become outdated before it is fully executed, while a single visual observation provides limited information about object motion. DynamicVLA introduces the Dynamic Object Manipulation (DOM) benchmark to evaluate manipulation under independently moving objects and changing environmental conditions [3]. The DOMINO benchmark further evaluates generalization across dynamic tasks, motion patterns, and robot embodiments [4].

Temporal alignment alone is insufficient for understanding object motion. In dynamic environments, the policy must reason not only about where an object is currently located, but also about its motion and likely future state. PUMA addresses this problem by incorporating historical optical flow and future object-centric feature prediction into the policy representation [4]. Historical optical flow provides explicit motion cues, while learnable world queries are trained to predict future object fea-

tures. This encourages the policy to anticipate object motion without requiring the future-prediction branch at inference time.

A.5 Adaptive Replanning and Policy Adaptation

A fixed execution horizon assumes that the same number of predicted actions should be executed throughout a task. In practice, different situations may require different execution horizons. Adaptive Action Chunking (AAC) uses uncertainty in the predicted action sequence to determine how long the current chunk should be executed [7]. High action uncertainty generally leads to shorter execution and earlier replanning, while low uncertainty permits longer execution.

Other approaches learn the execution horizon from task reward rather than relying on action uncertainty. Adaptive Action Chunking via Multi-Chunk Q Value Estimation (ACH) evaluates different prefixes of the same predicted chunk using learned Q-values [17]. Dynamic Execution Horizon Prediction (DEHP) instead learns a categorical policy that selects the execution horizon while keeping the underlying chunk generator frozen [10]. Both methods use reinforcement learning to adapt action commitment, but differ in whether they explicitly estimate the values of candidate prefixes or directly learn a horizon-selection policy.

A complementary direction is to adapt the generated behavior rather than the execution horizon. Diffusion Steering via Reinforcement Learning (DSRL) keeps a pretrained diffusion or flow generator frozen and learns to steer its sampling process through latent noise [6]. This allows reinforcement learning to select higher-reward behaviors without updating the underlying generative model. Real-Time EXPO-FT extends reinforcement-learning-based adaptation to online action correction by using a lightweight reactive policy to modify remaining actions based on updated observations [18].

B Implementation Details

B.1 Prediction and Execution Horizons

We train separate policies with different prediction horizons H to study the duration of the predicted trajectory. For execution-horizon comparisons, we keep the trained checkpoint and prediction horizon fixed and vary the number of actions consumed from each prediction. Checkpoint comparisons are reported separately because changing the trained policy can also change its response to frequent replanning.

B.2 Adaptive Action Chunking

AAC estimates action uncertainty from $N = 20$ action chunks sampled in parallel and selects an execution horizon using an entropy criterion and a minimum action-magnitude constraint [7]. Our implementation computes uncertainty over the Cartesian translation, rotation, and gripper components. Gaussian differential entropy is used for continuous actions, while categorical entropy is used for discrete gripper commands. The average entropy $\overline{E}_s$ is calculated over the first s actions.

The execution horizon is selected as

$$s^* = \max\left(\arg\max_s \left(\overline{E}_{s+1} - \overline{E}_s\right), \xi\right),$$

where ξ imposes a minimum action-magnitude constraint,

$$\xi = \min\{l \mid m(l) > \alpha\}.$$

Here, $m(l)$ combines accumulated translational displacement, rotational change, and gripper switching, while α denotes the movement threshold. In our real-robot experiments, $\alpha = 0.4$ achieved the best performance among the tested values.

After selecting s^*, the controller executes the corresponding portion of the predicted chunk before replanning. Sampling multiple candidate chunks increases inference latency, which is included in the evaluated execution pipeline.

B.3 DynamicVLA Execution Mechanisms

We transfer continuous inference and latency-aware action skipping from DynamicVLA [3] to the $\pi_{0.5}$ client-server pipeline. The client updates an observation mailbox while inference is running. Each returned chunk is indexed relative to the observation used to generate it, and actions corresponding to elapsed action intervals are discarded before execution.

Our implementation retains the $\pi_{0.5}$ architecture and current-observation input. It therefore differs from the complete DynamicVLA model, which uses a smaller backbone and temporal observations. We also introduce a minimum execution window W: a new prediction replaces the current chunk only after at least W actions from the current chunk have been executed. This modification limits the frequency of chunk switching and reduces how quickly new predictions can affect the robot.

B.4 FASTER

We implement FASTER [2] using horizon-aware sampling, action streaming, prefix conditioning, and early stopping. The server sends actions as they finish denoising. The client stores the received actions by index and executes them when required. Each request includes the actions expected to be executed during inference, which form the conditioned prefix of the next prediction.

The horizon-aware schedule uses $u_0 = 0.9$ and $\alpha = 0.6$, selected in preliminary real-robot experiments. Denoising stops once the conditioned prefix and the actions required for the selected execution horizon have been finalized.

We first implemented the inference-time components, then added mixed-schedule fine-tuning and training-time action conditioning. During fine-tuning, the horizon-aware and constant schedules are sampled with equal probability, $p = 0.5$. Training-time conditioning provides a clean action prefix while the remaining actions are noisy, exposing the policy to the prefix structure used during execution.

B.5 Real-Time Chunking

The initial RTC chunk is generated synchronously to populate the action buffer. Subsequent requests include the latest observation, the remaining previous prediction, and an estimate of the inference delay [1].

During denoising, RTC guides the new chunk toward the overlapping actions from the previous prediction. Actions expected to execute during inference receive the strongest guidance. The guidance weight decreases over the remaining overlap, while actions beyond the overlap are generated without this constraint. The correction requires a vector-Jacobian product at each denoising step.

The execution horizon is set to

$$s = \max(d, s_{\min}),$$

where $s_{\min}$ is the prescribed minimum horizon and d is updated from measured inference delays.

B.6 Cartesian ABPolicy Adaptation

ABPolicy [8] predicts B-spline control points and uses continuity-constrained refitting before executing a new trajectory. We retain the Cartesian action representation of our demonstrations.

Positions are fitted directly. Orientations are expressed relative to a reference quaternion q_{ref},

$$q_{\mathrm{rel},t} = q_{\mathrm{ref}}^{-1} \otimes q_t,$$

and mapped to rotation vectors before fitting. During training, the reference is taken from the present-time boundary of the trajectory window; during inference, it is the current robot orientation. Reconstructed rotation vectors are converted back to quaternions before execution.

We use $P = 8$ past actions and $H = 32$ future actions, giving a 40-action trajectory represented by $N = 8$ control points. During refitting, the first $N_{\mathrm{free}} = 4$ control points are optimized against the

recently executed actions; the remaining control points retain their predicted values. The reference contains both the P past actions and the actions executed during inference. A regularization term with weight 0.05 penalizes displacement of the last free control point from its predicted value.

B.7 PUMA and Offline Feature Caching

For PUMA [4], the client records a history of static-camera images. Four historical frames are sampled at a stride of four action steps, corresponding to $t-16$, $t-12$, $t-8$, and $t-4$. At $20\,\mathrm{Hz}$, this history spans $0.8\,\mathrm{s}$. Each historical frame is compared with the current frame to compute an optical-flow map, which is encoded alongside the current observations.

Four world queries receive supervision from object-centric features at $t+4$, $t+8$, $t+12$, and $t+16$. GroundingDINO localizes the target object, SAM2 segments it, and a frozen DINO encoder extracts the target feature. These features are generated once and stored in an offline cache. During training, the corresponding cached features supervise the world queries with a loss weight of 0.05.

The teacher models are used only to prepare training targets. Historical optical flow is computed during inference. Our PUMA implementation uses synchronous execution by default. We additionally evaluate an asynchronous execution variant to study the effect of inference scheduling.

B.8 DSRL with Temporal Visual History

We use DSRL-SAC [6] to train a latent-space actor and critic while keeping the fine-tuned VLA frozen. At each policy query, the actor selects a latent input w_t, which the VLA maps to an action chunk:

$$w_t \sim \pi_\phi^W(\cdot \mid z_t), \qquad A_t = G_\theta(o_t, \ell, p_t, w_t).$$

Here, z_t contains the steering policy's proprioceptive and visual features, and G_θ is the frozen VLA generator.

For each camera, three frames separated by three action steps are encoded independently using a shared convolutional encoder. A temporal convolution aggregates their features. The latest-frame feature is projected separately and added as a residual to retain the current spatial information. At $20\,\mathrm{Hz}$, consecutive sampled frames are separated by $0.15\,\mathrm{s}$, so the history spans $0.30\,\mathrm{s}$.

The actor and critic receive this temporal representation. The underlying VLA continues to receive its current observation and the selected latent input. Real-robot training uses a binary task-success reward supplied after each rollout. The execution horizon is fixed at $s = 10$ for the reported steering experiments.

C Experimental Setup and Evaluation Protocol

C.1 Base Policy and Training

We use $\pi_{0.5}$ as the base policy and perform full supervised fine-tuning on the demonstrations for each task. Training runs on NVIDIA A100 GPUs with an effective batch size of 128 for all configurations. We use three baseline checkpoints: the 1k and 5k checkpoints are trained with a prediction horizon of $H = 20$ and correspond to 1,000 and 5,000 fine-tuning steps, respectively, while the 1.5k checkpoint is trained with $H = 50$ for 1,500 steps.

C.2 Simulation Tasks

DOM. We construct a pick-and-place task based on DOM [3], with the instruction "pick up the lemon and place it into the red bowl." The initial positions of the lemon and bowl are randomized. The dataset contains 100 static demonstrations and 211 dynamic demonstrations, recorded at $25\,\mathrm{Hz}$. In the dynamic setting, the lemon receives an initial velocity uniformly sampled from $[0.05, 0.25]\,\mathrm{m/s}$. Actions are represented in Cartesian space. The camera study compares wrist and opposite views, side and opposite views, and all three views.



Figure 6: Manipulation tasks used in our experiments. (a) Dynamic pick-and-place in DOM. (b) Adjust Bottle in DOMINO. (c) Place Object into Basket in DOMINO.

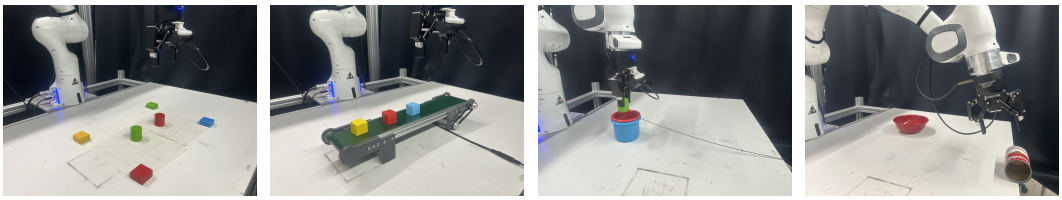


Figure 7: Real-robot manipulation tasks. (a) Static cylinder placement. (b) Conveyor-belt picking. (c) Moving-cup placement. (d) Rolling-can grasping.

DOMINO. We use the *Adjust Bottle* and *Place Object Basket* tasks from DOMINO [4] with the dual-arm Franka Panda embodiment. The simulator operates at 30 Hz and uses Cartesian actions. Each task contains three dynamic levels, with 50 clean and 200 randomized demonstrations per level. This gives 750 demonstrations per task and 1,500 across both tasks. Randomization varies background textures, lighting, table height, and tabletop clutter. Evaluation uses the clean environment at each dynamic level.

C.3 Real-Robot Tasks and Demonstrations

We conduct four real-robot manipulation tasks, including one static task and three dynamic tasks. All demonstrations are collected using a Franka Panda with a Robotiq gripper, a fixed third-person camera, and a wrist-mounted camera.

Static Cylinder Placement. The robot places a specified colored cylinder on a specified colored cube. Four cubes occupy fixed positions, while red and green cylinders vary within a central workspace region. The dataset contains 400 demonstrations, with 50 for each of eight language instructions specifying the cylinder and destination cube.

Conveyor Belt. The robot must grasp the red cube from a belt carrying three differently colored cubes and place it into a bowl. The dataset contains 60 demonstrations: 20 with a stationary belt, 20 at a slow speed, and 20 at a medium speed.

Moving Cup. The robot starts with a cylinder in its gripper and must place it into a cup. The dataset contains 40 static demonstrations with the cup at different positions and 75 dynamic demonstrations in which an operator pulls the cup using a rope. Pulling speed and direction vary across demonstrations.

Rolling Can. An operator rolls a can toward the robot, which must grasp it and place it into a bowl. The dataset contains 60 demonstrations collected at different rolling speeds.

C.4 Inference and Robot Control

The policy server runs on a remote A100 GPU node or an NVIDIA GeForce RTX 5090 workstation. The local robot client constructs observations, requests predictions, and buffers returned

actions. Predicted actions contain absolute Cartesian positions, quaternion orientations, and gripper commands. Demonstrations are recorded at $20\,\mathrm{Hz}$, while action execution frequencies of $20\,\mathrm{Hz}$ and $30\,\mathrm{Hz}$ are evaluated to investigate their influence on task performance.

The control layer runs at $100\,\mathrm{Hz}$. It reads the latest Cartesian target, limits excessively large position changes, and sends pose commands to the robot. Gripper commands are sent when the requested gripper state changes.

C.5 Real-Robot Evaluation Protocol

Method Comparisons. For the general method comparisons, each method is evaluated over 20 trials per task. Conveyor-belt trials vary belt speed, cube position, and cube ordering; stationary-belt trials are excluded from this comparison. Rolling-can evaluation contains ten slower and ten faster trials. Moving-cup evaluation varies movement speed and direction. The can and cup are moved manually, so their trajectories cannot be reproduced exactly across methods.

Conveyor-Belt Task Setup and Evaluation. We consider three conveyor-belt conditions: stationary, slow ($2.5\,\mathrm{cm/s}$), and fast ($5\,\mathrm{cm/s}$). The task involves three cubes, with the red cube as the target. In the stationary condition, the target cube can be placed at three positions along the conveyor belt (Left, Middle, and Right). In the dynamic conditions, the belt moves from left to right, and the cubes initially start on the left side. Here, Left, Middle, and Right refer to the red cube's relative position within the three-cube sequence.

For selected experiments, we evaluate nine conditions formed by three target positions and three belt conditions, with five trials per condition, giving 45 trials per configuration.

Targeted DSRL Evaluation. The original policy is evaluated over five trials per condition across the nine conveyor-belt conditions. After adaptation, the targeted condition receives 15 trials, while each of the eight remaining conditions receives five trials. We report success counts to make the unequal evaluation sizes explicit. When reporting a mean across conditions, we average the nine condition-specific success rates with equal weight. A pooled success rate instead weights conditions by their trial counts; these two aggregates are distinguished in the corresponding tables.

We evaluate noise levels of 0.5 and 2.5 for 5k checkpoints, but only 0.5 for 1k checkpoints, as higher noise induces random robot motions.

C.6 Execution Metrics

Time to First Action. TTFA is the interval between sending an observation to the policy server and receiving the first executable action:

$$\mathrm{TTFA} = t_{\mathrm{first}} - t_{\mathrm{request}}.$$

It includes inference and communication latency, as well as any waiting time imposed by the environment's update cycle within this request–response interval, and measures how soon an executable action from a new chunk becomes available. For streaming methods, the first action can arrive before generation of the full chunk is complete.

Task Completion Time. For successful trials, task completion time is measured from the start of inference to the first gripper release.

Inference Calls. Inference calls count the policy requests issued during a trial. It reflects the executed portion of the prediction, which may be shorter than the full prediction horizon.

In the real-robot method comparison, timing and inference-count statistics are computed over successful trials. Success rate uses all evaluation trials.

D Supplementary Experimental Results

D.1 Real-Robot Inference and Execution Metrics

Table 5: Inference calls and timing metrics for the real-robot static pick-and-place task.

Setting	s	IC	TTFA	TCT
$\pi_{0.5}$ Synchronous	20	7.0	0.295 s	8.264 s
	35	4.0	0.296 s	7.378 s
	50	3.0	0.261 s	6.991 s
$\pi_{0.5}$ with FASTER	5	27.5	**0.096 s**	6.728 s
	25	6.0	0.101 s	6.608 s
	40	4.0	0.108 s	6.709 s
$\pi_{0.5}$ with DynamicVLA	10	8.5	0.250 s	**6.019 s**
	20	6.5	0.260 s	6.280 s
	30	4.5	0.295 s	6.316 s
$\pi_{0.5}$ with RTC	10	13.8	0.339 s	6.853 s
	25	6.0	0.331 s	6.579 s
	40	4.0	0.322 s	6.588 s
$\pi_{0.5}$ with ABPolicy	6	20.4	0.244 s	6.754 s
$\pi_{0.5}$ with PUMA	15	9.7	0.320 s	9.584 s
	35	4.0	0.359 s	7.685 s
	40	3.0	0.343 s	7.372 s

Table 5 reports inference and execution metrics for the static pick-and-place task, with policy inference performed on an NVIDIA A100 GPU and a fixed prediction horizon of $H = 50$. We compare synchronous $\pi_{0.5}$ with FASTER, DynamicVLA, RTC, ABPolicy, and PUMA. Execution horizons are varied where applicable to evaluate their effects on inference calls, time to first action, task completion time.

Table 6: Inference calls and timing metrics for asynchronous $\pi_{0.5}$ on the dynamic conveyor-belt task.

Setting	s	IC	TTFA	TCT
$\pi_{0.5}$ Asynchronous	5	61.0	0.145 s	15.088 s
	10	22.5	0.144 s	10.952 s
	15	12.1	0.143 s	8.624 s
$\pi_{0.5}$ Asynchronous + AAC	Adaptive	12.1	0.285 s	7.436 s

Table 6 reports the same metrics for the dynamic conveyor-belt task, with inference performed on an NVIDIA RTX 5090 GPU. We evaluate asynchronous $\pi_{0.5}$ using the 5k checkpoint and a fixed prediction horizon of $H = 20$, comparing fixed execution horizons ($s = 5, 10, 15$) with Adaptive Action Chunking (AAC). Timing and inference-count metrics are averaged over successful trials.